

deep learning

O deep learning é um subconjunto do [aprendizado de máquina](#) orientado por [redes neurais](#) de várias camadas cujo design é inspirado na estrutura do cérebro humano. Modelos de deep learning servem de base para a maioria das [inteligências artificiais \(IA\)](#) de última geração atualmente, desde [computer vision](#) e [IA generativa](#) até carros autônomos e robótica.

Ao contrário da lógica matemática explicitamente definida dos [algoritmos de aprendizado de máquina](#) tradicionais, as redes neurais artificiais dos modelos de deep learning abrangem muitas camadas interconectadas de "neurônios", onde cada uma realiza uma operação matemática. Ao usar o aprendizado de máquina para ajustar a força das conexões entre neurônios individuais em camadas adjacentes, em outras palavras, os pesos e *vieses* variáveis do [modelo](#), a rede pode ser otimizada para produzir resultados mais precisos. Embora as redes neurais e o deep learning tenham se tornado inextricavelmente associados umas às outras, elas não são estritamente sinônimos: "deep learning" refere-se ao treinamento de modelos com pelo menos quatro camadas (embora as arquiteturas de redes neurais modernas sejam frequentemente muito "mais profundas" do que isso) .

É essa estrutura distribuída, altamente flexível e ajustável que explica o incrível poder e a versatilidade do deep learning. Imagine dados de treinamento como pontos de dados espalhados em um gráfico bidimensional e o objetivo do [treinamento de modelo](#) é encontrar uma linha que passe por cada um desses pontos de dados. Essencialmente, o aprendizado de máquina tradicional visa realizar isso usando uma única função matemática que produz uma única linha (ou curva); O deep learning, por outro lado, pode reunir um número arbitrário de linhas menores e individualmente ajustáveis para formar a forma desejada. Redes neurais são *aproximadores universais*: foi provado teoricamente que, para qualquer função, existe um arranjo de redes neurais que pode reproduzi-la.¹

Os modelos de deep learning são mais comumente treinados por meio de [aprendizado supervisionado](#) em [dados rotulados](#) para executar tarefas de regressão

e [classificação](#). Porém, como as redes neurais de grande escala geralmente exigem uma enorme quantidade de [dados de treinamento](#) para alcançar o desempenho, o custo e o trabalho de adquirir conjuntos de dados suficientemente grandes de exemplos de treinamento anotados podem ser proibitivos. Isso levou ao desenvolvimento de técnicas para replicar tarefas de aprendizado supervisionado usando dados *não rotulados*. O termo [aprendizado autossupervisionado](#) foi cunhado por Yann LeCun no final da década de 2010 para eliminar a ambiguidade desses métodos do [aprendizado tradicional não supervisionado](#). Desde então, o aprendizado autossupervisionado surgiu como um modo proeminente de treinamento de redes neurais, especialmente para os [modelos de base](#) que sustentam a IA generativa.

Embora as redes neurais (ou conceitos análogos) tenham sido introduzidas por [cientistas de dados](#) no início da história do aprendizado de máquina, seu avanço não começou para valer até o final dos anos 2000 e início dos anos 2010. O advento de redes de deep learning na maioria dos subconjuntos de aprendizado de máquina foi possibilitado em parte pelos avanços nas unidades de processamento gráfico de alto desempenho ([GPUs](#)), que possibilitaram o processamento paralelo de grandes quantidades de etapas computacionais. Como o deep learning exige um enorme poder de computação tanto para [treinamento](#) quanto para [inferência](#), esses avanços de hardware aumentaram consideravelmente a velocidade e a praticidade da implementação de modelos de deep learning em escala.

Como funciona o deep learning

As redes neurais artificiais são, em termos gerais, inspiradas no funcionamento dos [circuitos neurais](#) do cérebro humano, cujo funcionamento é impulsionado pela complexa transmissão de sinais químicos e elétricos através de redes distribuídas de células nervosas (neurônios). No deep learning, os "sinais" análogos são as saídas ponderadas de muitas operações matemáticas aninhadas, cada uma realizada por um "neurônio" artificial (ou *nó*), que compõem coletivamente a rede neural.

Em resumo, um modelo de deep learning pode ser entendido como uma série intrincada de equações aninhadas que mapeiam uma entrada para uma saída. Ajustar a influência relativa de equações individuais dentro dessa rede usando

processos especializados de aprendizado de máquina pode, por sua vez, alterar a maneira como a rede mapeia entradas para saídas.

Embora esse framework seja muito poderoso e versátil, ele prejudica a *interpretabilidade*. Frequentemente, há pouca ou nenhuma explicação intuitiva, além de uma explicação matemática bruta, sobre como os valores de parâmetros de modelos individuais aprendidos por uma rede neural refletem as características do mundo real. Por esse motivo, os modelos de deep learning são frequentemente chamados de “caixas-pretas”, especialmente quando comparados aos tipos tradicionais de modelos de aprendizado de máquina informados pela [engenharia de funcionalidades](#) manual.

Em relação às técnicas clássicas de aprendizado de máquina, o deep learning requer uma quantidade extremamente grande de dados e recursos computacionais para treinamento. Dado o custo e a complexidade do hardware de nível empresarial necessário para desenvolver e implementar aplicações sofisticadas de deep learning, os serviços de computação em nuvem se tornaram uma parte cada vez mais integral do ecossistema de deep learning.

Estrutura de redes neurais profundas

As redes neurais artificiais compreendem camadas interconectadas de "neurônios" artificiais (ou *nós*), cada um dos quais realiza sua própria operação matemática (chamada de "[função de ativação](#)"). Existem muitas funções de ativação diferentes; uma rede neural geralmente incorpora múltiplas funções de ativação dentro de sua estrutura, mas normalmente todos os neurônios em uma determinada camada da rede neural serão configurados para executar a mesma função de ativação. Na maioria das neural networks, cada neurônio da *camada de entrada* é conectado a cada um dos neurônios da camada seguinte, que estão conectados aos neurônios da camada seguinte, e assim por diante.

A saída da ativação de cada nó contribui com parte da entrada fornecida para cada um dos nós da camada seguinte. Crucialmente, as funções de ativação realizadas em cada nó são *não lineares*, permitindo às redes neurais modelar padrões e dependências complexos. É o uso de funções de ativação não lineares que

distingue uma rede neural profunda de um modelo de regressão linear (muito complexo).

Embora algumas arquiteturas especializadas de redes neurais, como [combinações de modelos especializados](#) ou [redes neurais convolucionais](#), impliquem variações, adições ou exceções a esse arranjo, todas as redes neurais empregam alguma versão dessa estrutura central. O número específico de camadas, o número de nós dentro de cada camada e as funções de ativação escolhidas para os nós de cada camada são [hiperparâmetros](#) a serem determinados manualmente antes do treinamento.

Cada uma dessas inúmeras conexões neurais a neurônio é multiplicada por um *peso* único, que amplifica (ou diminui) a influência de cada conexão. A entrada fornecida para a função de ativação de cada neurônio pode ser entendida como a *soma ponderada* das saídas de cada neurônio na camada anterior. Geralmente também há um termo de *viés* único adicionado a cada função de ativação, que funciona de forma semelhante ao termo de viés de uma função de regressão comum.

Durante o treinamento, a rede neural "aprende" por meio de ajustes em cada um desses pesos e termos de viés que produzem produções mais precisas. Estes são os [parâmetros](#) do modelo: quando você lê sobre, por exemplo, um [grande modelo de linguagem \(LLM\)](#) com 8 bilhões de "parâmetros", esse número reflete cada conexão neurônio a neurônio ponderada e viés específico de neurônio na rede neural do modelo.

As camadas intermediárias, chamadas de *camadas ocultas* da rede, são onde ocorre a maior parte do aprendizado. É a inclusão de *múltiplas* camadas ocultas que distingue um modelo de deep learning de uma "não profunda" rede neural, como uma [máquina de Boltzmann restrita \(RBN\)](#) ou um perceptron multicamadas padrão (MLP). A presença de várias camadas ocultas permite que um modelo de deep learning aprenda funcionalidades hierárquicas complexas dos dados, com camadas anteriores identificando padrões mais amplos e camadas mais profundas identificando funcionalidades mais granulares.

Para realizar a [inferência](#), a rede completa uma *passagem para a frente*: a camada de entrada recebe dados de entrada, geralmente na forma de uma [embedding de vetores](#), com cada neurônio de entrada processando uma funcionalidade individual do vetor de entrada. Por exemplo, um modelo que funciona com imagens em escala de cinza de 10x10 pixels normalmente terá 100 neurônios em sua camada de entrada, com cada neurônio de entrada correspondendo a um pixel individual. Portanto, as Neural networks normalmente exigem que as entradas sejam fixadas a um determinado tamanho, embora técnicas de pré-processamento, como pooling ou normalização, possam fornecer alguma flexibilidade em relação ao tamanho dos próprios dados de entrada originais.

Os dados são progressivamente transformados e transmitidos para os nós de cada camada subsequente até a última camada. As funções de ativação dos neurônios na camada de saída calculam a previsão de saída final da rede. Por exemplo, cada nó de saída de um [modelo de classificação](#) profunda pode executar uma [função softmax](#), que essencialmente pega uma entrada numérica e a dimensiona para uma probabilidade, entre 0 e 1, de que a entrada pertença a uma categoria de classificação potencial. Em seguida, o modelo geraria a categoria correspondente a qualquer nó de saída que tivesse gerado a maior saída.

Treinamento de redes neurais profundas

Embora o potencial teórico das redes neurais profundas sempre fosse aparente, não se sabia inicialmente como treinar redes neurais de forma eficiente. O objetivo de otimizar os parâmetros do modelo por meio do treinamento é reduzir o erro das saídas finais da rede, mas isolar e calcular separadamente como *cada um* dos milhares, senão milhões ou bilhões, de pesos interconectados de uma rede neural contribuíram para o erro geral é completamente impraticável.

Esse obstáculo foi superado pela introdução de dois algoritmos essenciais: [retropropagação](#) e [gradiente descendente](#).

Retropropagação

[Retropropagação](#), abreviação de “propagação reversa de erros”, é um método elegante para calcular como as mudanças em qualquer peso ou viés em uma rede neural afetarão a precisão das previsões do modelo.

Lembre-se de que uma rede neural artificial é essencialmente uma série de funções matemáticas aninhadas: as produções dos neurônios de uma camada servem como entradas para os neurônios da próxima camada e assim por diante. Durante o treinamento, essas equações interconectadas são aninhadas em ainda outra função: uma *função de perda*, que mede a diferença média (ou “perda”) entre a saída desejada (ou “*verdade fundamental*”) para uma determinada entrada e a saída real da rede neural para cada passagem para a frente.

Depois que os hiperparâmetros iniciais do modelo tiverem sido determinados, o treinamento normalmente começa com uma inicialização aleatória dos parâmetros do modelo. O modelo faz previsões em um lote de exemplos do conjunto de dados de treinamento e a função de perda rastreia o erro de cada previsão. O objetivo do treinamento é otimizar iterativamente os parâmetros até que a perda média tenha sido reduzida para abaixo de um limite aceitável.

A retropropagação envolve uma única *passagem para trás* de ponta a ponta através da rede, começando com a saída da função de perda e trabalhando até a camada de entrada. Usando a *regra da cadeia de cálculo*, a retropropagação calcula o “gradiente” da função de perda: um vetor de derivadas parciais da função de perda em relação a cada variável em cada equação que, em última análise, se aninha no cálculo da função de perda. Em outras palavras, descreve como aumentar ou diminuir a saída da função de ativação de qualquer neurônio individual afetará a perda geral — que, por extensão, descreve como quaisquer *alterações em qualquer um dos pesos dessas produções são multiplicadas* (ou nos termos de viés adicionados a essas saídas) aumentará ou diminuirá a perda.

Gradiente descendente

O gradiente calculado durante a retropropagação serve como entrada para um algoritmo de *gradiente descendente*.

Mover para baixo —*descer*—o gradiente da função de perda diminuirá a perda (e, assim, aumentará a precisão). Como o gradiente calculado durante a retropropagação contém as derivadas parciais da função de perda em relação a cada parâmetro do modelo, sabemos em qual direção “mover” o valor de cada parâmetro para reduzir a perda.

Cada etapa envolve uma atualização dos parâmetros do modelo e reflete o "aprendizado" do modelo com seus dados de treinamento. Nosso objetivo é atualizar iterativamente os pesos até atingir o gradiente mínimo. O objetivo dos algoritmos de gradiente descendente é encontrar os ajustes de parâmetros específicos que "descerão" o gradiente de forma mais eficiente.

Implementação de modelos de deep learning

Há uma série de frameworks de código aberto para desenvolver modelos de deep learning, seja treinando um modelo do zero ou [realizando o ajuste fino](#) um [modelo pré-treinado](#). Essas [bibliotecas de aprendizado de máquina](#) oferecem uma variedade de módulos pré-configurados e fluxo de trabalho para construir, treinar e avaliar redes neurais, simplificar e otimizar o processo de desenvolvimento

Entre os frameworks de código aberto mais populares para trabalhar com algoritmos de deep learning estão [PyTorch](#), TensorFlow e (particularmente para LLMs) a biblioteca Hugging Face Transformers. É recomendado aprender [Python](#) antes de trabalhar com esses frameworks.

Tipos de modelos de deep learning

Apesar de sua potência e potencial inerentes, o desempenho adequado em determinadas tarefas continua sendo impossível ou impraticável para redes neurais profundas convencionais ("vanilla"). As últimas décadas testemunharam várias inovações na arquitetura de redes neurais padrão, cada uma visando um desempenho aprimorado em tarefas e tipos de dados específicos.

Vale a pena notar que um determinado tipo de rede neural pode se prestar a vários tipos de *modelos* de deep learning e vice-versa. Por exemplo, um modelo de codificador automático usado para tarefas de imagem pode aproveitar uma arquitetura baseada em redes neurais; os modelos de difusão podem utilizar arquiteturas baseadas em CNN ou em transformadores.

Redes neurais convolucionais (CNNs)

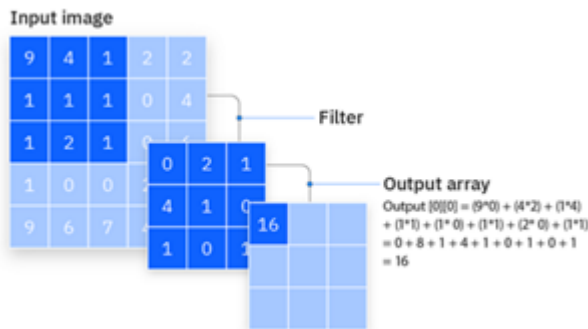
[As redes neurais convolucionais \(CNNs\)](#) são principalmente (mas não exclusivamente) associadas a tarefas [de visão computacional](#), como [detecção](#) de objetos, [reconhecimento de](#) imagens, [classificação de](#) imagens e [segmentação de](#) imagens, pois se destacam no reconhecimento de padrões "locais" (como relações entre pixels adjacentes em um imagem).

A intuição por trás do desenvolvimento das CNNs foi a de que, para determinadas tarefas e modalidades de dados, como classificar imagens de alta resolução com centenas ou milhares de pixels, redes neurais de tamanho suficiente, compreendendo apenas camadas padrão e totalmente conectadas, teriam parâmetros demais para generalizar bem novos dados pós-treino. Em outras palavras, eles seriam computacionalmente ineficientes e propensos a **overfitting** dos dados de treinamento, em vez de aprender padrões do mundo real genuinamente úteis.

Em teoria, redes neurais que podem detectar funcionalidades distintas e outras funcionalidades significativas poderiam economizar poder computacional ao extrair essas funcionalidades da imagem bruta para processamento adicional (e descartando informações sobre regiões da imagem sem funcionalidades significativas). Uma maneira de fazer isso seria usar *filtros*: pequenas matrizes bidimensionais de números cujos valores correspondem à forma de funcionalidades úteis. Por exemplo, os valores de um filtro que verifica os pixels de uma imagem em busca dos cantos superiores esquerdos podem ter esta aparência:

[1010101010100000100000100000100000]

Agora imagine esse filtro 5x5 sendo multiplicado por uma grade 5x5 de pixels em uma imagem de entrada. Na linguagem matemática, isso é chamado de **convolução**: uma operação matemática em que uma função modifica (ou *convolui*) uma segunda função. Se os valores de pixel forem semelhantes aos do filtro, o produto dessa multiplicação (o *produto escalar*) será grande, e a funcionalidade que esses pixels representam será capturada; caso contrário, o produto escalar será pequeno e os pixels serão ignorados.



Uma pequena seção dos valores de pixel de uma imagem (à esquerda) é multiplicada por um filtro de convolução (meio), resultando em uma representação de dimensão inferior dos pixels originais (à direita), que reflete como os pixels originais se assemelham às informações representadas pelo filtro.

As CNNs adicionam *camadas de convolução*, contendo muito menos nós do que camadas totalmente conectadas padrão que atuam como esses filtros. Em vez de exigir um nó único (com um peso único) correspondente a cada pixel individual na imagem, o filtro de uma camada de convolução avança ao longo de toda a imagem, processando uma grade de pixels de tamanho correspondente de cada vez. Isso não só extrai informações úteis, mas também reduz significativamente o número de parâmetros de modelo exclusivos necessários para processar toda a imagem.

As CNNs são tipicamente muito "profundas" (em termos de número de camadas) do que as redes neurais padrão, mas, como as camadas de convolução contêm relativamente poucos neurônios, ainda são eficientes em termos de contagem total de parâmetros. Conforme os dados atravessam a CNN, cada camada convolucional extrai funcionalidades progressivamente mais granulares, montando um "mapa de funcionalidades". A funcionalidade final é eventualmente passada para uma camada padrão totalmente conectada que realiza as previsões finais. No treinamento, o modelo naturalmente aprende pesos para as camadas de convolução que resultam em seus filtros capturando funcionalidades propícias a resultados precisos.

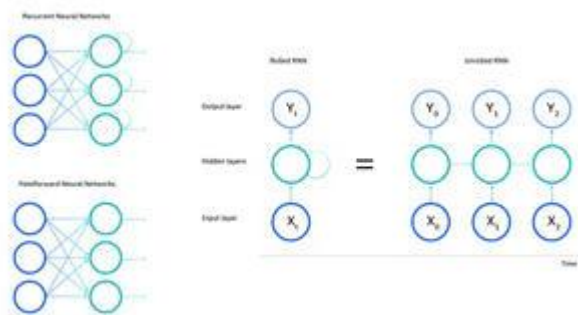
Redes neurais recorrentes (RNNs)

As [redes neurais recorrentes \(RNNs\)](#) são usadas em tarefas que envolvem dados sequenciais, como forecasting, [reconhecimento de fala](#) ou [processamento de linguagem natural \(NLP\)](#).

Enquanto as redes neurais feedforward convencionais mapeiam uma única entrada para uma única saída, as RNNs mapeiam uma *sequência* de entradas para uma saída operando em um loop recorrente no qual a saída para uma determinada etapa

na sequência de entrada serve como entrada para o cálculo para o seguinte: . Na verdade, isso cria uma "memória" interna de entradas anteriores, chamada de *estado oculto*. Atualizado após cada etapa de tempo, esse estado oculto permite que uma RNN mantenha um entendimento do contexto e da ordem.

Embora a noção de uma única camada "agregada" seja útil para entender o conceito, essa recorrência também pode ser entendida como dados atravessando uma série de múltiplas camadas que compartilham pesos idênticos.



Uma RNN, exibida em formas "enrolada" e

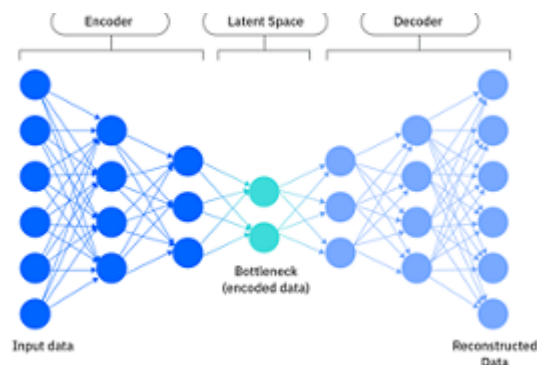
"desenrolada"

Isso leva a algumas deficiências fundamentais das RNNs convencionais, especialmente no que diz respeito ao treinamento. Lembre-se de que a retropropagação calcula o gradiente da função de perda, que determina como cada parâmetro do modelo individual deve aumentar ou diminuir. Quando cada uma dessas atualizações de parâmetros é repetida em muitas camadas recorrentes "idênticas", essas atualizações escalam exponencialmente: ampliar parâmetros pode levar a um *gradiente de explosão*, e minimizar parâmetros pode levar a *gradientes de desaparecimento*. Ambos os problemas podem introduzir instabilidade no treinamento, retardar o treinamento ou até mesmo interrompê-lo completamente. As RNNs padrão são, portanto, limitadas ao processamento de sequências relativamente curtas.

Vários aprimoramentos na arquitetura básica de RNNs, como redes de memória de curto prazo (LSTM) ou unidades recorrentes fechadas (GRUs), mitigam esses problemas e aumentam a capacidade do modelo de modelar dependências de longo alcance.

Autocodificadores

Os [autocodificadores](#) são projetados para compactar (ou *codificar*) dados de entrada e, em seguida, reconstruir (decodificar) a entrada original usando essa representação compactada. No treinamento, eles são otimizados para minimizar a *perda de reconstrução*: a divergência entre o ponto de dados reconstruído e os dados de entrada originais. Embora esse tipo de deep learning utilize dados não rotulados e dados não estruturados, os codificadores automáticos são geralmente considerados um exemplo arquetípico de [aprendizado autossupervisionado](#).



Em essência, isso força o modelo a aprender pesos que resultam na representação comprimida retendo apenas o subconjunto mais essencial e significativo das funcionalidades dos dados de entrada. No jargão de aprendizado de máquina, os codificadores automáticos modelam o [espaço latente](#).

Os autocodificadores têm uma variedade de casos de uso, como compactação de dados, [redução de dimensionalidade](#), [extração de funcionalidades](#), remoção de ruído de dados corrompidos e detecção de fraude.

Na maioria dos casos, a rede do decodificador serve apenas para ajudar a treinar o codificador e é descartada após o treinamento. Em [autocodificadores variacionais \(VAEs\)](#), um tipo de [modelo generativo](#), o decodificador é retido e usado para gerar novos pontos de dados, adicionando algum ruído aleatório às representações latentes aprendidas pelo codificador antes da reconstrução.

Modelos de transformador

O advento dos [modelos de transformação](#), apresentado pela primeira vez em um artigo seminal de 2017 do Google DeepMind intitulado "[Attention is all you Need](#)" ([PDF](#)), foi um momento divisor de águas no deep learning que levou diretamente à era atual da IA generativa.

Assim como as RNNs, os transformadores são inerentemente projetados para trabalhar com dados sequenciais. A funcionalidade central dos modelos de transformação é seu [mecanismo de autoatenção](#), do qual os modelos de transformação derivam sua impressionante capacidade de detectar os relacionamentos (ou dependências) entre cada parte de uma sequência de entrada. Mais importante ainda, esse mecanismo de atenção permite que os transformadores foquem seletivamente (ou "atendem") as partes de uma sequência de entrada que são mais relevantes em um determinado momento.

Mecanismos de atenção foram introduzidos pela primeira vez nos contextos de RNNs usadas para [tradução automática](#). Mas, diferentemente das RNNs, os transformadores não usam camadas recorrentes; uma arquitetura de transformação padrão usa apenas camadas de atenção e camadas feedforward padrão, aproveitando uma estrutura inovadora inspirada na lógica dos [bancos de dados relacionais](#).

Os transformadores são mais comumente associados a grandes modelos de linguagem (LLMs) e, por associação, a casos de uso de NLP, como geração de texto, [chatbots](#) e [análise de sentimento](#). Mas são modelos extremamente versáteis capazes de processar qualquer modalidade de dados sequenciais, incluindo dados de áudio ou [séries temporais](#). Até mesmo modalidades de dados como dados de imagem podem ser processadas por transformadores de visão (ViTs) por meio de soluções alternativas inteligentes para representá-los como uma sequência.

Embora os modelos de transformação tenham produzido resultados de última geração em quase todos os domínios da deep learning, eles não são necessariamente a escolha ideal para todo e qualquer caso de uso. Por exemplo, embora as ViTs tenham alcançado as melhores classificações de desempenho em benchmarks para tarefas de computer vision, as CNNs são significativamente mais rápidas e mais eficientes em termos computacionais. Para tarefas como detecção de objetos ou segmentação de imagens, a escolha entre um transformador ou CNN geralmente se resume a se uma determinada aplicação de deep learning deve priorizar a precisão máxima ou o feedback em tempo real.

Modelos Mamba

Lançados pela primeira vez em 2023, os [modelos Mamba](#) são uma nova arquitetura de deep learning para dados sequenciais. Derivado de uma variação de [modelos de espaço de estado \(SSMs\)](#), o Mamba tem conexões teóricas interessantes com RNNs, CNNs e modelos de transformadores. Mais importante ainda, o Mamba compartilha com os transformadores a capacidade de priorizar seletivamente (ou descartar) informações passadas com base em sua relevância em um determinado momento, embora com um mecanismo totalmente único para fazer isso.

Até o momento, o Mamba é talvez a única arquitetura a rivalizar significativamente com os transformadores no domínio dos LLMs, oferecendo desempenho comparável com eficiência computacional significativamente maior devido ao seu algoritmo que consome muito menos memória.

Redes adversárias generativas (GANs)

Assim como os VAEs, as [redes adversárias generativas \(GANs\)](#) são redes neurais usadas para criar novos dados semelhantes aos dados de treinamento originais. As GANs são uma arquitetura conjunta que combina duas redes de deep learning treinadas *adversamente* em um jogo de soma zero.

A rede do **gerador** cria novos pontos de dados, como imagens originais. Qualquer arquitetura generativa capaz de produzir a saída desejada pode ser usada para uma rede de geradores de GANs. Sua única característica definidora é como ele interage com o discriminador, e seu único requisito é que o algoritmo seja diferenciável (e, portanto, capaz de ser otimizado por meio de retropropagação e gradiente descendente).

O **discriminador** recebe imagens “reais” do conjunto de dados e imagens “falsas” produzidas pelo gerador e é encarregado de determinar se uma determinada imagem é real ou falsa. Assim como o gerador, o discriminador pode assumir a forma de qualquer arquitetura adequada.

Primeiro, o discriminador é treinado para classificar corretamente imagens falsas. Durante esse período, os pesos do gerador estão congelados.

Em seguida, os pesos do discriminador são congelados e o feedback do discriminador é usado para treinar o gerador. Os pesos do gerador são otimizados para produzir imagens com maior probabilidade de enganar o discriminador.

O processo é repetido: o discriminador recebe outra variedade de imagens “reais” de dados de treinamento e imagens “falsas” do gerador – que agora são, presumidamente, mais convincentes. O discriminador mais uma vez prevê se cada imagem é real ou falsa e é atualizado mais uma vez.

Mais uma vez, o feedback do discriminador (presumivelmente mais difícil de enganar) é usado para treinar ainda mais o gerador.

O processo continua iterativamente até que o discriminador não consiga mais discernir entre amostras reais e falsas.

As GANs são capazes de aprender a produzir exemplos incrivelmente precisos, mas a natureza adversária do processo torna o treinamento inerentemente complicado e instável.

Modelos de difusão

Os [modelos de difusão](#) estão entre as arquiteturas de rede neural mais proeminentes em [IA generativa](#). Eles são práticos e de alto desempenho, oferecem a estabilidade de treinamento dos VAEs e a fidelidade de saída dos GANs. Eles são mais comumente usados para geração de imagens, mas também são capazes de gerar dados de texto, vídeo e áudio.

Assim como os codificadores automáticos, os modelos de difusão são essencialmente treinados para destruir uma imagem e depois reconstruí-la com precisão, embora de uma maneira completamente diferente. No treinamento, os modelos de difusão aprendem a *difundir* gradualmente um ponto de dados passo a passo com ruído gaussiano e, em seguida, *reverter* esse processo para reconstruir a entrada original. Ao fazer isso, eles ganham a capacidade de gerar novas amostras (semelhantes aos dados de treinamento originais) “reduzindo ruído” de uma amostra de ruído aleatório.

Os *modelos de difusão latente* são essencialmente um híbrido de VAEs e modelos de difusão: eles primeiro comprimem (codificam) os dados de entrada para o espaço latente, em seguida, realizam o processo de difusão e, em seguida, alimentam o resultado para um decodificador que o aumenta para o tamanho de imagem desejado.

Embora os modelos de difusão normalmente usem uma arquitetura baseada em CNN - especificamente, a arquitetura de *rede U* usada de forma proeminente para segmentação em imagens médicas — alguns aproveitam uma arquitetura baseada em transformador.

Redes neurais de grafos

As **redes neurais de gráficos (GNNs)** são projetadas para tarefas que exigem a modelagem de relacionamentos mais complexos entre diferentes entidades do que o típico na maioria das modalidades de dados.

Considere os dados de imagem, nos quais os pixels de uma imagem são organizados em uma grade bidimensional: qualquer pixel está diretamente conectado a, no máximo, oito pixels adjacentes. Uma CNN padrão é adequada para modelar essas relações. Mas esse recurso não funciona bem para modelar os relacionamentos dentro, por exemplo, de uma rede de redes sociais na qual um determinado usuário pode estar conectado diretamente a milhares de outros usuários e indiretamente a muitos milhares mais.

A estrutura das redes neurais de gráficos permite representações de dados mais complexas e irregulares do que são possíveis no fluxo unidirecional de dados inerente a outras arquiteturas de redes neurais.

FONTE: <https://www.ibm.com/br-pt/think/topics/deep-learning>